

**“Año de la Esperanza y el Fortalecimiento de la
Democracia”**



UPLA

TEMA:

SEMANA 01

CURSO: ALGORITMOS Y ESTRUCTURA DE DATOS

DOCENTE: FERNÁNDEZ BEJARANO RAUL ENRIQUE

INTEGRANTES: CONDORI MORENO JOSHEP E.

FACULTAD: INGENIERIA

E.A.P: INGENIERÍA DE SISTEMAS Y COMPUTACIÓN

2026

1. Método para eliminar un elemento "x"

```
// 1. Método para eliminar un elemento "x"
public void eliminar(int x) {
    int index = -1;
    for (int i = 0; i < elementos.length; i++) {
        if (elementos[i] == x) {
            index = i;
            break;
        }
    }
    if (index != -1) {
        int[] nuevoArreglo = new int[elementos.length - 1];
        for (int i = 0, k = 0; i < elementos.length; i++) {
            if (i == index) continue;
            nuevoArreglo[k++] = elementos[i];
        }
        elementos = nuevoArreglo;
        System.out.println("Elemento " + x + " eliminado.");
    } else {
        System.out.println("Elemento no encontrado.");
    }
}
```



Método 1

Eliminar 'x'



Busca un valor dentro de un arreglo y lo elimina, creando un nuevo arreglo con un elemento menos.



¿Qué hace?

El **método** recorre el arreglo para encontrar el elemento **x**. Si lo encuentra, crea un nuevo arreglo de tamaño **N-1** y copia todos los elementos excepto el que se desea eliminar.



Idea principal

"Encuentro x, creo un arreglo más pequeño y copio todo menos x."



Ejemplo

Eliminar el valor **3** del siguiente arreglo:

```
int[] elementos = {5, 8, 3, 7, 9};
```

Antes



Se elimina

Después



Paso a paso

1

Buscar x

Recorremos el arreglo para encontrar el índice del elemento.

```
int indice = -1;
for (int i = 0; i < elementos.length; i++) {
    if (elementos[i] == x) {
        indice = i;
        break;
    }
}
```



Si **x = 3**,
se encuentra
en el índice 2.

2

Verificar si existe

Comprobamos si se encontró el valor en el arreglo.

```
if (indice != -1) {
    // El elemento existe
}
```



Si **indice = -1**,
significa que
el valor no está
en el arreglo.

3

Crear un arreglo nuevo

Como vamos a eliminar un elemento, el nuevo arreglo tendrá **N - 1** posiciones.

```
int[] nuevo = new int[elementos.length - 1];
```



Si el arreglo original tenía 5 elementos,
el nuevo tendrá 4.

4

Copiar los elementos

Copiamos todos los elementos excepto el que queremos eliminar.

```
int j = 0;
for (int i = 0; i < elementos.length; i++) {
    if (i != indice) {
        nuevo[j] = elementos[i];
        j++;
    }
}
```



El índice **j** se usa
para llenar el nuevo
arreglo sin dejar
espacios vacíos.



Resumen visual

Arreglo original



N = 5

Se elimina
el 3



Nuevo arreglo



N = 4



Importante

En Java, un arreglo (**int[]**) tiene tamaño fijo. Por eso no podemos simplemente borrar una posición; debemos crear otro arreglo más pequeño.



En resumen

Buscar x



Verificar



Crear N - 1



Copiar sin x

2. Método para modificar un elemento "x"

```
// 2. Método para modificar un elemento "x"
public void modificar(int x, int nuevoValor) {
    for (int i = 0; i < elementos.length; i++) {
        if (elementos[i] == x) {
            elementos[i] = nuevoValor;
            System.out.println("Elemento " + x + " modificado a " + nuevoValor);
            return;
        }
    }
    System.out.println("Elemento no encontrado para modificar.");
}
```



MÉTODO 2

Modificar 'x'

Busca un valor dentro de un arreglo y lo reemplaza por uno nuevo, manteniendo el mismo tamaño.



¿Qué hace?

El método recorre el arreglo para encontrar la posición del elemento **x**. Al encontrarlo, reemplaza su valor directamente por **nuevoValor** y detiene la búsqueda.



Idea principal

"Busco **x**, encuentro su posición y reemplazo su valor directamente."

Ejemplo

Modificar el valor **3** por **10** en el siguiente arreglo:

```
int[] elementos = {5, 8, 3, 7, 9};
```

Antes

5 8 3 7 9

3 se modifica por 10

Después

5 8 10 7 9

Paso a paso del algoritmo

1 Recorrido (Búsqueda)

Iniciamos un ciclo for para examinar secuencialmente cada elemento.

```
for (int i = 0; i < elem.length; i++) { if (elem[i] == x) { // Coincidencia } }
```

Recorre desde el índice 0 hasta N-1

2 Coincidencia

Comprobamos si el elemento coincide con el buscado.

```
if (elementos[i] == x) {
```

Detiene la búsqueda al hallar la coincidencia.

Resumen visual del cambio en memoria



Importante

A diferencia de **Eliminar x**, el método **Modificar x** NO crea un arreglo nuevo.

El arreglo conserva exactamente la misma cantidad de elementos (N=5); solamente cambia el valor almacenado en la casilla encontrada.

✓ En resumen

1. Recorrer (for)

→ 2. Comparar (if == x)

→ 3. Asignar nuevoValor

→ 4. Finalizar (return)

3. Método para insertar un elemento "x" en una posición "y"

```
// 3. Método para insertar un elemento "x" en una posición "y"
public void insertar(int x, int y) {
    if (y < 0 || y > elementos.length) {
        System.out.println("Posición inválida.");
        return;
    }
    int[] nuevoArreglo = new int[elementos.length + 1];
    for (int i = 0, k = 0; i < nuevoArreglo.length; i++) {
        if (i == y) {
            nuevoArreglo[i] = x;
        } else {
            nuevoArreglo[i] = elementos[k++];
        }
    }
    elementos = nuevoArreglo;
    System.out.println("Elemento " + x + " insertado en la posición " + y);
}
```



MÉTODO 3

Insertar 'x' en posición 'y'

Inserta un valor en un índice específico, desplazando el resto y creando un nuevo arreglo más grande.



¿Qué hace?

Valida que la posición **y** sea válida. Crea un nuevo arreglo de tamaño **N+1** e inserta el valor **x** en el índice **y**, desplazando los demás elementos mediante un puntero auxiliar **k**.

Idea principal

"Valido la posición, creo un arreglo con N+1 casillas e inserto x en y desplazando los elementos."

</> Ejemplo

Insertar el valor **x = 99** en la posición **y = 2**:

```
int[] elementos = {5, 8, 3, 7, 9};
```

Antes

5	8	3	7	9
---	---	---	---	---

↓ Inserta 99 en posición y=2

Después

5	8	99	3	7	9
---	---	----	---	---	---

Paso a paso del algoritmo

1 Validar posición

Verifica que el índice **y** no sea menor a 0 ni mayor a la longitud actual.

```
if (y < 0 || y > elem.length) { System.out.println("Inválida"); return; }
```

Evita `ArrayIndexOutOfBoundsException`

2 Crear nuevo N+1

Instancia un arreglo con un espacio extra para nuevo elemento.

```
int[] nuevo = new int[elem.le
```

Si media 5, el nuevo mide 6

Resumen visual del cambio en memoria

Arreglo original

5	8	3	7	9
---	---	---	---	---

N = 5

Inserta en y=2
→

Nuevo arreglo

5	8	99	3	7	9
---	---	----	---	---	---

N = 6 (Un elemento más)

Importante

En Java, un arreglo (`int[]`) es de tamaño fijo.

Por ello, no podemos simplemente "empujar" casillas para abrir espacio; es imprescindible crear un nuevo arreglo de tamaño **N + 1** y reorganizar la copia usando un índice auxiliar **k**.

✓ En resumen

Validar 'y'



Crear N + 1



Insertar 'x' en 'y'



Copiar elem[k++]

4. Método para sumar los elementos del arreglo

```
// 4. Método para sumar los elementos del arreglo
public int sumarElementos() {
    int suma = 0;
    for (int num : elementos) {
        suma += num;
    }
    return suma;
}
```



MÉTODO 4

Sumar Elementos

Recorre todo el arreglo acumulando el valor de cada elemento y retorna el total resultante.



 ¿Qué hace?

Inicializa una variable acumuladora **suma** en 0. Luego utiliza un bucle *for-each* para visitar cada número del arreglo e ir sumándolo consecutivamente al total.

 Idea principal

"Empiezo en 0, recorro elemento por elemento y voy acumulando el total."

 Ejemplo

Calcular la suma total del arreglo:

```
int[] elementos = {5, 8, 3, 7, 9};
```

5

+

8

+

3

+

7

+

9

Total obtenido: 32

 Paso a paso del algoritmo

1 Inicializar

Se crea la variable acumuladora **suma** asignándole el valor inicial de 0.

```
int suma = 0;
```

Base limpia para acumular

2 Bucle For-Each

Recorre automáticamente cada número **num** almacenado dentro del arreglo.

```
for (int num : elementos) { suma += num; }
```

Visita directo cada valor

3 Retornar Total

Al finalizar el recorrido completo, devuelve el valor final acumulado.

```
return suma;
```

Regresa la suma calculada

 Seguimiento de la acumulación

Paso	Elemento (num)	Operación	Suma Acumulada
Inicio	-	<code>suma = 0</code>	0
1	5	<code>0 + 5</code>	5
2	8	<code>5 + 8</code>	13
3	3	<code>13 + 3</code>	16
4	7	<code>16 + 7</code>	23
5	9	<code>23 + 9</code>	32

 Importante

Ventaja del For-Each: El sintaxis `for (int num : elementos)` es más limpia y previene errores con los índices, ya que no requiere manejar la propiedad `.length` ni subíndices `i`.

No modifica la estructura: Tampoco altera el contenido ni el tamaño original del arreglo.

 En resumen

1. suma = 0 → 2. Iterar con for-each → 3. suma += num → 4. return suma

5. Método para hallar el número menor, mayor y el promedio

```
// 5. Método para hallar el número menor, mayor y el promedio
public void hallarMenorMayorPromedio() {
    if (elementos.length == 0) return;

    int menor = elementos[0];
    int mayor = elementos[0];
    double suma = 0;

    for (int num : elementos) {
        if (num < menor) menor = num;
        if (num > mayor) mayor = num;
        suma += num;
    }

    double promedio = suma / elementos.length;
    System.out.println("Menor: " + menor + " | Mayor: " + mayor + " | Promedio: " + promedio);
}
```



MÉTODO 5

Hallar Menor, Mayor y Promedio

Analiza un arreglo en un solo recorrido para extraer los valores extremos y calcular la media aritmética.





¿Qué hace?

Verifica que el arreglo no esté vacío. Luego inicializa **menor** y **mayor** con el primer elemento (`elementos[0]`) y **suma** en 0.0. Recorre el arreglo actualizando los límites y calculando el promedio final.



Idea principal

"En una sola pasada comparo extremos y acumulo para promediar."



</> Ejemplo

Dado el siguiente arreglo de enteros:

```
int[] elementos = {12, 5, 20, 8};
```

12

5

20

8

MENOR
5

MAYOR
20

PROMEDIO
11.25



Paso a paso del algoritmo

1

Validar e Inicializar

Si está vacío, sale con `return`. Si no, toma el primer valor como pivote inicial.

```
if (elementos.length == 0) return; int menor = elementos[0]; int mayor = elementos[0]; double suma = 0;
```



Evita errores de arreglo vacío

2

Ex...



Prueba de escritorio (Arreglo: {12, 5, 20, 8})

num	num < menor	num > mayor	suma	Estado (Men / May)
-	-	-	0.0	12 / 12 (Inicio)
12	No (12<12)	No (12>12)	12.0	12 / 12
5	Sí (5<12)	No	17.0	5 / 12
20	No	Sí (20>12)	37.0	5 / 20
8	No	No	45.0	5 / 20
Promedio = 45.0 / 4 = 11.25				



Puntos clave de sintaxis

1. Control de Arreglo Vacío:

La línea `if (elementos.length == 0) return;` evita una excepción `ArrayIndexOutOfBoundsException` al intentar acceder a `elementos[0]`.

2. Precisión Decimal:

La variable `suma` es de tipo `double` para garantizar que la división `suma / elementos.length` devuelva un resultado decimal exacto.



En resumen

1. Validar vacíos

→

2. Menor/Mayor = [0]

→

3. Iterar y actualizar

→

4. Promedio = suma / N

6. Método para suma de Impares

```
// 6. Método para suma de Impares
public int sumaImpares() {
    int suma = 0;
    for (int num : elementos) {
        if (num % 2 != 0) {
            suma += num;
        }
    }
    return suma;
}
```

MÉTODO 6



Suma de Impares

Filtra los elementos del arreglo evaluando si son números impares para acumular únicamente su valor.

1

2

3

4

 **¿Qué hace?**

Inicializa el acumulador **suma** en 0. Recorre el arreglo mediante un bucle *for-each* y, utilizando el operador módulo %, comprueba si el residuo de dividir cada número entre 2 es distinto de 0 ($\text{num \% } 2 \neq 0$). Si cumple, lo suma.

 **Idea principal**
"Evalúo cada número: si no es par (residuo $\neq 0$), lo agrego al total acumulado."

 **Ejemplo**

Sumar solo los elementos impares del arreglo:

```
int[] elementos = {4, 7, 2, 9, 5, 6};
```

4

7

2

9

5

6

Suma de impares (7 + 9 + 5): 21

⚙ Paso a paso del algoritmo

1 Inicializar

Declara la variable local **suma** e inicia su valor en 0 para comenzar el conteo.

```
int suma = 0;
```

Acumulador limpio en cero

2 Filtrar con Módulo

La condición ($\text{num \% } 2 \neq 0$) identifica si el número actual tiene residuo al dividirse por 2.

```
for (int num : elementos) { if (num \% 2 != 0) { suma += num; } }
```

⚡ Ignora números pares

3 Retorna Resultado

Una vez finalizada la iteración, devuelve el valor correspondiente a la suma de los números impares filtrados.

Devuelve el resultado filtrado

 **Prueba de escritorio (Arreglo: {4, 7, 2, 9, 5, 6})**

num	num % 2	¿Es Impar? ($\neq 0$)	Acción	Suma
4	0	No	Ignorar	0
7	1	Sí	Suma 0 + 7	7
2	0	No	Ignorar	7
9	1	Sí	Suma 7 + 9	16
5	1	Sí	Suma 16 + 5	21
6	0	No	Ignorar	21

 **Operador Módulo (%) en Java**
¿Cómo funciona $\text{num \% } 2$?
El operador % calcula el **residuo** de una división entera:

- $7 \% 2 = 1 \rightarrow$ **Impar** (residuo $\neq 0$). Se acumula.
- $4 \% 2 = 0 \rightarrow$ **Par** (residuo = 0). Se descarta.

Sintaxis equivalente:
Para sumar números pares se utilizaría la condición opuesta: ($\text{num \% } 2 == 0$).

✓ En resumen

1. suma = 0

→

2. num % 2 != 0

→

3. suma += num

→

4. return suma

7. Método para mover el Primer elemento a la Última posición

```
// 7. Método para mover el Primer elemento a la Última posición
public void moverPrimerAlUltimo() {
    if (elementos.length <= 1) return;

    int primerElemento = elementos[0];
    for (int i = 0; i < elementos.length - 1; i++) {
        elementos[i] = elementos[i + 1];
    }
    elementos[elementos.length - 1] = primerElemento;
    System.out.println("Primer elemento movido al final.");
}
```

MÉTODO 7



Mover Primer Elemento al Final



Efectúa una rotación a la izquierda desplazando todos los elementos una posición atrás y colocando el primero al final.

¿Qué hace?

Guarda el valor inicial del índice 0 en una variable auxiliar. Luego recorre el arreglo moviendo cada elemento a la izquierda (`elementos[i] = elementos[i+1]`). Finalmente, inserta el valor guardado en la última posición disponible (`elementos[elementos.length - 1]`).

Idea principal

"Resguardo la cabeza, desplazo todo un espacio hacia la izquierda y pongo la cabeza al final."

</> Ejemplo

Estado Inicial vs. Estado Final tras la rotación:

Antes:

10 20 30 40

Desplazamiento a la izquierda

Después:

20 30 40 10

⚙ Paso a paso del algoritmo

1 Validación y Respaldo

Si tiene 0 o 1 elemento, no hace nada. Si no, resguarda el primer valor antes de sobrescribirlo.

```
if (elementos.length <= 1) return; int primerElemento = elementos[0];
```

Preserva el valor inicial

2 Desplazar a la Izquierda

Copia cada elemento posterior en la posición anterior.

```
for (int i = 0; i < elementos.length - 1; i++) {
```

Avanza una posición atrás

👁 Prueba de escritorio (Arreglo: {10, 20, 30, 40})

Paso / i	Operación	Estado del Arreglo
Inicio	Respaldo <code>primerElemento = 10</code>	{ 10, 20, 30, 40 }
i = 0	<code>elem[0] = elem[1]</code> (copia 20)	{ 20, 20, 30, 40 }
i = 1	<code>elem[1] = elem[2]</code> (copia 30)	{ 20, 30, 30, 40 }
i = 2	<code>elem[2] = elem[3]</code> (copia 40)	{ 20, 30, 40, 40 }
Final	<code>elem[3] = primerElemento (10)</code>	{ 20, 30, 40, 10 }

⚠ Puntos clave de control

1. Límite del bucle (`length - 1`):

Se debe iterar hasta `i < elementos.length - 1` para evitar un error `ArrayIndexOutOfBoundsException` al evaluar `elementos[i + 1]`.

2. Caso base / Arreglos pequeños:

Si el arreglo tiene 0 o 1 elementos (`length <= 1`), no hay nada que desplazar y el método termina inmediatamente con `return`.

✓ En resumen

1. Guardar `elem[0]`



2. `elem[i] = elem[i+1]`



3. `elem[last] = temp`



4. Imprimir aviso

8. Método para mover el Primer elemento Par al Final del arreglo

```
// 8. Método para mover el Primer elemento Par al Final del arreglo
public void moverPrimerParAlFinal() {
    if (elementos.length <= 1) return;

    int indexPar = -1;
    for (int i = 0; i < elementos.length; i++) {
        if (elementos[i] % 2 == 0) {
            indexPar = i;
            break;
        }
    }

    if (indexPar != -1) {
        int primerPar = elementos[indexPar];
        for (int i = indexPar; i < elementos.length - 1; i++) {
            elementos[i] = elementos[i + 1];
        }
        elementos[elementos.length - 1] = primerPar;
        System.out.println("Primer número par movido al final.");
    } else {
        System.out.println("No hay números pares en el arreglo.");
    }
}

public void mostrarArreglo() {
    System.out.println(Arrays.toString(elementos));
}
}
```

MÉTODO 8



Mover Primer Par al Final

Localiza la posición del primer número par en el arreglo y lo traslada a la última casilla mediante desplazamiento.



¿Qué hace?

Busca secuencialmente el primer número par ($\text{num} \% 2 == 0$) y guarda su posición en `indexPar` deteniendo la búsqueda con `break`. Si encuentra un par ($\text{indexPar} != -1$), desplaza los elementos desde ese punto hacia la izquierda y coloca el par al final.

Idea principal

"Encuentro el primer par, abro espacio recorriendo los elementos siguientes a la izquierda y lo ubico al final."

</> Ejemplo

Traslado del primer número par (8) al final:

Antes (Primer par en índice 1):

5 8 3 9

← Desplazamiento desde el índice del par

Después:

5 3 9 8

Paso a paso del algoritmo

1 Buscar el Primer Par

Inicia `indexPar = -1`. Recorre con un ciclo hasta hallar un valor con residuo 0. Guarda el índice y rompe el ciclo.

```
int indexPar = -1; for (int i = 0; i < elementos.length; i++) { if (elementos[i] % 2 == 0) { indexPar = i; break; }
```

🔍 Frena al primer hallazgo

Prueba de escritorio (Arreglo: (5, 8, 3, 9))

Fase	Detalle / Operación	Estado del Arreglo
Búsqueda	En $i = 1: 8 \% 2 == 0 \rightarrow \text{indexPar} = 1$	{ 5, 8, 3, 9 }
Resguardo	$\text{primerPar} = 8$	{ 5, 8, 3, 9 }
$i = 1$	$\text{elem}[1] = \text{elem}[2]$ (copia 3)	{ 5, 3, 3, 9 }
$i = 2$	$\text{elem}[2] = \text{elem}[3]$ (copia 9)	{ 5, 3, 9, 9 }
Final	$\text{elem}[3] = \text{primerPar}$ (8)	{ 5, 3, 9, 8 }

Método Auxiliar e Instrucción Break

• ¿Por qué usar break?

Garantiza que solo guardemos el índice del **primer** número par encontrado, ignorando pares posteriores.

• Método de apoyo `mostrarArreglo()`:

Utiliza `Arrays.toString(elementos)` para imprimir de forma rápida y legible la estructura actual del arreglo en consola.

```
public void mostrarArreglo() { System.out.println(Arrays.toString(elementos)); }
```

✓ En resumen

1. Buscar par + break



2. if ($\text{indexPar} != -1$)



3. Desplazar desde `indexPar`



4. `elem[last] = primerPar`